

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)		
Student Name:	SANTHOSH R	Reg.No.:	192312450
Department:	ECE	Date:	21.08.26

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I Souhash K / (192312450) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Souhash K
Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)	
Q1	4	6	3	3	2	18
Q2	4	5	3	3	2	17
Q3	5	4	4	2	1	16
Q4	4	5	3	2	2	16
Q5	3	6	4	3	2	18
						<u>85</u>

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

DESIGN AND SIMULATE REGISTER TRANSFER AND ALU OPERATIONS

1. Aim

To design and simulate a basic processor datapath that performs register transfer operations and arithmetic and logical operations using multiple registers and an Arithmetic Logic Unit (ALU).

2. Objectives

- Understand the purpose of processor registers.
- Represent register transfers using Register Transfer Language (RTL).
- Perform arithmetic operations using the ALU.
- Perform logical AND and OR operations.
- Understand the role of the control unit and control signals.
- Observe the sequence of data movement and computation.

3. Theory

A register is a group of flip-flops used to store a binary word temporarily. Registers are much faster to access than main memory and are therefore used to hold operands, addresses, instructions, and intermediate results.

The ALU is the main computational block. Depending on the control input, it can perform operations such as addition, subtraction, AND, OR, comparison, and other arithmetic or logical functions. The control unit selects the required operation and coordinates data movement.

4. Proposed Datapath

The proposed datapath consists of four registers, a common data path, an ALU, and control signals. Data is transferred from a source register to the ALU, processed, and then written into a destination register.

Conceptual flow: Source Register → Bus → ALU → Bus → Destination Register

5. Registers Used

Register	Initial Value	Purpose
R1	10	First operand
R2	5	Second operand
R3	0	Destination/result register
R4	0	Temporary register

6. Register Transfer Language

Register Transfer Language is a symbolic notation used to describe micro-operations. $R1 \rightarrow R3$ means that the contents of R1 are transferred to R3. $R3 \leftarrow R3 + R4$ means that the ALU adds R3 and R4 and stores the result in R3.

7. Detailed Operation

Step	Micro-operation	Explanation	Output
1	$R1 \rightarrow R3$	Copy first operand	$R3 = 10$
2	$R2 \rightarrow R4$	Copy second operand	$R4 = 5$
3	$R3 \leftarrow R3 + R4$	ALU addition	$R3 = 15$
4	$R3 \leftarrow R3 - R4$	ALU subtraction	$R3 = 10$
5	$R3 \text{ AND } R4$	Logical AND	0
6	$R3 \text{ OR } R4$	Logical OR	15

8. Binary Verification

10 in binary is 1010 and 5 in binary is 0101. Therefore, $1010 \text{ AND } 0101 = 0000$, which gives 0. Similarly, $1010 \text{ OR } 0101 = 1111$, which gives decimal 15.

9. Control Signals

- Source-select signal chooses the register that places data on the bus.
- Destination-enable signal allows the selected register to load data.
- ALU-select signal chooses ADD, SUB, AND, or OR.
- Clock signal controls when the register transfer takes place.
- Write-enable controls storage of the final result.

10. Experimental Procedure

1. Initialize R1 with 10 and R2 with 5.
2. Transfer R1 to R3 and verify the value.
3. Transfer R2 to R4 and verify the value.
4. Select ADD in the ALU and store the result in R3.
5. Select SUB and verify the subtraction result.
6. Apply AND and OR operations and observe the output.
7. Record all intermediate values in a result table.

11. Expected Result and Analysis

The experiment should produce correct values for all register transfers and ALU operations. The result confirms that data movement and computation are controlled by the combination of registers, datapath, ALU, and control signals. Incorrect control selection can lead to an incorrect destination or operation, showing why coordinated control is necessary.

12. Applications

- CPU datapath design.
- Arithmetic instruction execution.
- Digital processor simulation.
- Embedded processor architecture.
- Educational processor design.

13. Result

Register transfer and ALU operations were successfully designed and analyzed. The experiment demonstrates the interaction between registers, datapath, ALU, and control unit.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

SINGLE-BUS AND MULTI-BUS ORGANIZATION

1. Aim

To design a computer organization using single-bus and multi-bus structures and compare their data-transfer efficiency, execution time, and resource utilization.

2. Theory

A bus is a shared communication path used to transfer data, addresses, or control information. In a single-bus organization, several components share one path. In a multi-bus organization, two or more buses are used to permit greater parallelism.

3. Single-Bus Organization

A single-bus processor is relatively simple because all registers and functional units communicate through a common bus. However, only one major source can normally drive the shared bus at a time, so complex operations may require several clock cycles.

Example: $R3 \leftarrow R1 + R2$ requires the processor to make both operands available to the ALU and then transfer the result to R3. The shared bus may become a bottleneck.

4. Multi-Bus Organization

A multi-bus organization provides independent paths for operands and results. A three-bus arrangement can allow one bus to carry the first operand, another bus to carry the second operand, and a third bus to carry the ALU result.

Conceptual flow: $R1 \rightarrow \text{Bus A} \rightarrow \text{ALU}$; $R2 \rightarrow \text{Bus B} \rightarrow \text{ALU}$; $\text{ALU} \rightarrow \text{Bus C} \rightarrow R3$.

5. Comparison

Parameter	Single Bus	Multi Bus
Hardware complexity	Low	High

Number of buses	One	Two or more
Parallelism	Low	High
Data transfer	Mostly sequential	More parallel
Execution time	Higher	Lower
Cost	Lower	Higher
Bottleneck	More likely	Reduced
Control complexity	Lower	Higher
Performance	Moderate	High

6. Memory and I/O Transfers

In addition to register transfers, buses connect the processor with memory and I/O devices. A single shared path can become congested when instruction fetch, data access, and I/O operations compete for the same resource. Multiple buses can separate or overlap some of these transfers.

7. Timing Analysis

Assume a single-bus operation takes 4 cycles and an equivalent multi-bus operation takes 2 cycles.

Time reduction = $4 - 2 = 2$ cycles.

Percentage reduction = $(2/4) \times 100 = 50\%$.

This example shows how parallel paths can reduce transfer time.

8. Advantages

- Single bus: simple, economical, easy to control, and suitable for basic designs.
- Multi bus: higher throughput, more parallel transfers, reduced bottlenecks, and improved ALU utilization.

9. Experimental Procedure

1. Create a register set and common bus for the single-bus model.
2. Perform register-to-register transfer operations.
3. Measure the number of clock cycles.
4. Create a two-bus or three-bus model.
5. Perform the same operations using parallel paths.
6. Record execution cycles and compare the results.
7. Analyze the trade-off between hardware complexity and performance.

10. Result and Analysis

The multi-bus architecture provides better data-transfer efficiency because multiple operands can be transferred in parallel. The single-bus architecture remains attractive when simplicity and lower hardware cost are more important than maximum performance.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

FIVE-STAGE INSTRUCTION PIPELINE

1. Aim

To simulate a five-stage instruction pipeline and compare its execution time, speedup, and throughput with a non-pipelined processor.

2. Theory

Pipelining divides instruction execution into stages so that different instructions can occupy different stages at the same time. It is similar to an assembly line: while one instruction is being executed, another can be decoded and a third can be fetched.

3. Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Fetch instruction from memory
ID	Instruction Decode	Decode instruction and identify operands
EX	Execute	Perform ALU or address calculation
MEM	Memory	Perform memory read/write when required
WB	Write Back	Store result in destination register

4. Example Instructions

Use four independent instructions: I1 = ADD, I2 = SUB, I3 = AND, and I4 = OR.

5. Timing Table

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM
8				WB

6. Performance Calculation

For a k-stage pipeline executing n instructions under ideal conditions:

Pipeline cycles = $k + n - 1$

For $k = 5$ and $n = 4$: $5 + 4 - 1 = 8$ cycles.

Without pipelining, four instructions requiring five stages each need $4 \times 5 = 20$ cycles.

Speedup = $20/8 = 2.5$.

7. Throughput

Throughput is the number of completed instructions per unit time. After the pipeline is filled, an ideal five-stage pipeline can complete approximately one instruction per clock cycle. Actual throughput is lower when stalls, hazards, cache misses, or branch penalties occur.

8. Pipeline Efficiency

For this simple example, an idealized efficiency can be expressed as speedup divided by the number of stages: $2.5/5 = 0.5$, or 50%. This is an example for the small instruction set and should not be interpreted as a universal processor efficiency.

9. Advantages and Limitations

- Advantages: higher throughput, better utilization of functional units, and overlapping instruction execution.
- Limitations: data dependencies, branch instructions, structural conflicts, additional hardware, and control complexity.

10. Experimental Procedure

1. Define the five pipeline stages.
2. Enter a sequence of independent instructions.
3. Simulate instruction movement through IF, ID, EX, MEM, and WB.
4. Record the cycle in which each instruction completes.
5. Repeat the experiment without pipelining.
6. Calculate execution time and speedup.
7. Compare throughput and explain the observed difference.

11. Result

The five-stage pipeline successfully overlaps instruction execution and reduces the total number of cycles for multiple instructions under ideal conditions. The calculated speedup for four independent instructions is 2.5.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

PIPELINE HAZARDS AND HAZARD RESOLUTION

1. Aim

To develop a pipeline scenario containing data, control, and structural hazards and analyze stalling, forwarding, and branch prediction techniques.

2. Theory

A pipeline hazard is a condition that prevents the next instruction from executing in its intended cycle. Hazards reduce the ideal throughput of a pipeline and may require stalls, forwarding, prediction, or additional hardware.

3. Data Hazard

A data hazard occurs when an instruction requires a value that is produced by an earlier instruction.

Example: I1: ADD R1, R2, R3; I2: SUB R4, R1, R5. Instruction I2 depends on the new value of R1.

4. Forwarding

Forwarding sends a newly generated result directly from an execution or later pipeline stage to a dependent instruction. It reduces the need to wait for the normal register write-back stage.

5. Stalling

A stall inserts an idle cycle into the pipeline. It is used when a dependency cannot be resolved immediately. Although simple, stalls reduce throughput.

6. Control Hazard

A control hazard occurs when a branch or jump changes the next instruction address. Until the branch decision is known, the processor may not know which instruction should be fetched.

Branch prediction attempts to predict whether a branch will be taken or not taken. A correct prediction avoids much of the branch delay, while an incorrect prediction requires the wrong-path instructions to be removed.

7. Structural Hazard

A structural hazard occurs when two instructions need the same hardware resource at the same time. For example, two operations may compete for a single memory port or execution unit.

8. Hazard Comparison

Hazard	Cause	Typical Effect	Resolution
Data	Dependency	Waiting for operand	Forwarding / Stalling
Control	Branch or jump	Uncertain next instruction	Branch prediction / Flush
Structural	Resource conflict	Resource contention	Extra resource / Stall

9. Performance Example

If an ideal pipeline completes a sequence in 8 cycles and two stall cycles are inserted, the actual execution becomes 10 cycles. The additional two cycles represent the performance penalty caused by hazards.

10. Experimental Procedure

1. Create dependent instructions to produce a data hazard.
2. Observe the hazard without forwarding.
3. Enable forwarding and compare the number of stalls.
4. Insert a conditional branch to create a control hazard.
5. Apply a simple branch prediction strategy.
6. Create a resource conflict to demonstrate a structural hazard.
7. Add or share resources and observe the change in execution time.
8. Compare the ideal and actual pipeline performance.

11. Analysis

The experiment demonstrates that a pipeline cannot achieve its theoretical maximum throughput in every situation. Data forwarding reduces data dependency delays, branch prediction reduces control penalties, and additional resources reduce structural conflicts. Effective hazard handling is therefore essential for processor performance.

12. Result

Data, control, and structural hazards were identified and suitable resolution techniques were analyzed. The results show that hazard management directly affects pipeline throughput and execution time.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

SUPERSCALAR, OUT-OF-ORDER, SPECULATIVE EXECUTION AND SMT

1. Aim

To analyze advanced processor execution techniques and evaluate how they improve instruction throughput and resource utilization.

2. Theory

Modern processors use instruction-level and thread-level parallelism to execute more work without simply increasing the clock frequency. Superscalar execution, out-of-order execution, speculative execution, and simultaneous multithreading are important techniques used to exploit this parallelism.

3. Superscalar Execution

A superscalar processor contains multiple execution units and can issue multiple independent instructions in the same clock cycle. For example, independent ADD, SUB, and AND operations may be assigned to separate execution resources.

4. Out-of-Order Execution

Out-of-order execution allows ready instructions to execute before earlier instructions that are waiting for data or resources. This prevents execution units from remaining idle.

Example: if I1 is waiting for memory while I2 and I3 are ready, the processor may execute I2 and I3 first, subject to dependency and correctness constraints.

5. Speculative Execution

Speculative execution predicts a future control path and begins executing instructions before the branch decision is finalized. If the prediction is correct, useful work has already been performed. If it is wrong, speculative work is discarded and the correct path is resumed.

6. Simultaneous Multithreading

SMT allows multiple threads to share the execution resources of one processor core. When one thread cannot use some resources because it is waiting, another thread may use those resources, improving overall utilization.

7. Hyper-Threading

Hyper-threading is a form of simultaneous multithreading in which one physical processor core is exposed as multiple logical processors. It enables multiple threads to share core resources.

8. Comparison

Technique	Primary Purpose	Main Benefit	Trade-off
Superscalar	Multiple instructions per cycle	Higher throughput	More hardware
Out-of-order	Execute ready instructions first	Better resource use	Complex scheduling
Speculative	Predict control flow	Lower branch penalty	Wrong predictions waste work
SMT	Multiple threads per core	Higher utilization	Resource contention
Hyper-threading	Logical processors per core	Better multithreaded use	Shared resources

9. Combined Operation

Modern processor designs can use these techniques together. SMT supplies multiple instruction streams; superscalar execution provides multiple execution resources; out-of-order scheduling selects ready instructions; and speculative execution predicts branches. The combined effect is increased parallelism and better utilization.

10. Experimental Scenario

1. Create a sequence containing independent arithmetic instructions.
2. Measure execution with one execution unit.
3. Repeat with multiple execution units to represent superscalar execution.
4. Introduce a long-latency instruction and observe out-of-order scheduling.
5. Introduce a branch and compare execution with and without prediction.
6. Run two independent instruction streams to demonstrate SMT.
7. Compare instruction throughput and resource utilization.

11. Analysis

Superscalar execution improves the number of instructions that can be processed per cycle. Out-of-order execution keeps resources busy when some instructions are delayed. Speculative execution reduces the visible cost of branches when predictions are accurate. SMT increases utilization by allowing more than one thread to compete for available execution resources.

The main limitation is complexity. These techniques require additional scheduling logic, prediction mechanisms, storage, and execution resources. They can also increase power consumption and create resource contention.

12. Result

The advanced execution techniques were successfully analyzed. The study demonstrates how modern processors increase throughput through instruction-level parallelism, prediction, dynamic scheduling, and multithreading.